

**LAPORAN TUGAS AKHIR UAS PEMROGRAMAN BERBASIS
OBJEK
RANCANG BANGUN SISTEM INFORMASI MANAJEMEN
RESERVASI HOTEL**



Disusun oleh:

- | | |
|--------------------------------|------------|
| 1. Rizan Abki Chaerullah | 3337240076 |
| 2. Muhammad Akbar Muzaki | 3337240019 |
| 3. Muhammad Fahriz Abiliansyah | 3337240087 |

**JURUSAN INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS SULTAN AGENG
TIRTAYASA**

2026

ABSTRAK

Sistem informasi reservasi pada industri perhotelan memegang peranan krusial dalam keberlangsungan operasional harian. Proses manual sering kali menyebabkan kendala berupa kesalahan pencatatan nomor kamar, ketidaksinkronan status kamar, serta ketidakefisienan pemesanan yang berpotensi memicu double booking. Penelitian tugas akhir ini bertujuan untuk merancang dan membangun sebuah aplikasi berskala desktop-console yaitu Hotel Reservation Management System menggunakan bahasa pemrograman Java. Metode pengembangan perangkat lunak yang diimplementasikan berfokus pada kualitas kode dan kemudahan pemeliharaan melalui penerapan konsep Pemrograman Berbasis Objek (OOP) secara penuh, pemisahan tanggung jawab dengan Data Access Object (DAO) Pattern, pengaturan logika bisnis pada Service Layer, kepatuhan terhadap prinsip SOLID (SRP dan DIP), serta jaminan keamanan transaksi database menggunakan PreparedStatement dan Try-With-Resources untuk koneksi database MySQL. Hasil dari penelitian ini membuktikan bahwa arsitektur berlapis (3-tier) mampu meningkatkan keamanan sistem dari ancaman SQL Injection sekaligus menjaga agar perangkat lunak fleksibel untuk dikembangkan lebih lanjut (maintainable) di masa yang akan datang.

BAB I: PENDAHULUAN

1.1 Latar Belakang Masalah

Di era digitalisasi saat ini, efisiensi operasional menjadi penentu daya saing utama pada berbagai sektor industri, termasuk jasa perhotelan. Salah satu aspek vital yang menentukan kualitas pelayanan hotel adalah kelancaran proses transaksi reservasi kamar. Reservasi bukan sekadar pencatatan nama tamu, melainkan sebuah siklus integrasi data kompleks yang melibatkan manajemen persediaan kamar secara real-time, klasifikasi tipe kamar, penyesuaian tarif inap, hingga pemantauan status fisik kamar (seperti pemeliharaan atau perbaikan).

Banyak hotel skala menengah ke bawah di Indonesia yang masih mengandalkan pencatatan berbasis kertas atau aplikasi mandiri sederhana yang tidak terstruktur dengan baik. Hal ini memicu rentetan kendala operasional yang serius:

1. Double Booking (Tumpang Tindih Reservasi): Tamu yang berbeda memesan satu nomor kamar yang sama pada rentang waktu yang berhimpitan akibat tidak adanya pembatasan dinamis pada aplikasi.
2. Ketidadaan Riwayat Terpadu: Sulit melacak status check-in, check-out, serta pembatalan secara terorganisir.
3. Kerentanan Keamanan Data: Query SQL yang dibangun secara konvensional sering kali mengabaikan validasi parameter masukan, membuka celah keamanan fatal berupa serangan SQL Injection.
4. Keterikatan Kode Tinggi (High Coupling): Perubahan pada struktur database memaksa perubahan total pada kode antarmuka, yang meningkatkan biaya pemeliharaan dan risiko kegagalan sistem.

Melalui mata kuliah Pemrograman Berbasis Objek (PBO), mahasiswa dibekali keahlian untuk mengatasi kendala rekayasa perangkat lunak tersebut menggunakan paradigma pemrograman berorientasi objek. Oleh karena itu, tugas akhir ini difokuskan untuk membangun aplikasi Hotel Reservation Management System berbasis Java Console. Program ini dirancang dengan arsitektur berlapis yang memisahkan logika antarmuka, aturan bisnis, dan akses data persisten MySQL.

1.2 Rumusan Masalah

Berdasarkan latar belakang masalah yang telah diuraikan, rumusan masalah dalam penelitian ini adalah sebagai berikut:

1. Bagaimana mengimplementasikan konsep Pemrograman Berbasis Objek (OOP) secara komprehensif pada aplikasi manajemen hotel?
2. Bagaimana cara mengaplikasikan pola desain Data Access Object (DAO) dan Service Layer guna mereduksi tingkat ketergantungan antarkomponen kode (low coupling)?
3. Bagaimana menjamin aspek keamanan data dari serangan SQL Injection pada saat melakukan transaksi ke MySQL database menggunakan JDBC?
4. Bagaimana memastikan efisiensi pemakaian memori database melalui pemanfaatan fitur-fitur modern pada Java SE?

1.3 Batasan Masalah

Agar pembahasan dalam laporan tugas akhir ini lebih terarah dan spesifik, diberikan beberapa batasan masalah sebagai berikut:

1. Antarmuka Pengguna: Menggunakan Java Console (CLI/Command Line Interface) dengan visualisasi tabel data dan navigasi menu berbasis angka masukan keyboard.
2. Sistem Database: Menggunakan database relasional MySQL yang berjalan lokal di lingkungan server Laragon versi terbaru.
3. Ruang Lingkup Fitur:
 - Pengelolaan kamar: Tambah kamar baru, lihat daftar kamar, pembaruan data kamar, penghapusan kamar, dan pencarian kamar berdasarkan nomor.
 - Pengelolaan reservasi: Pembuatan transaksi reservasi baru, tampilan seluruh transaksi, pencarian detail transaksi berdasarkan ID, pembaruan status transaksi (Booked, Check-in, Check-out, Cancelled), pembatalan, dan pencarian reservasi berdasarkan nama tamu.
4. Aturan Validasi: Validasi nama tamu tidak boleh kosong, validasi rentang tanggal check-out wajib setelah check-in, penolakan kamar berstatus terisi/maintenance, dan pembaruan otomatis status kamar ketika transaksi reservasi dilakukan.

1.4 Tujuan Penelitian

Adapun tujuan yang ingin dicapai melalui pengerjaan proyek tugas akhir ini adalah:

1. Menghasilkan aplikasi program Java Console manajemen hotel yang andal dengan validasi logika bisnis yang ketat.
2. Mengimplementasikan arsitektur berlapis (3-tier) dengan memisahkan UI, Service Layer, dan Data Access Object (DAO) Pattern secara bersih.
3. Menerapkan asas-asas desain berorientasi objek yang fleksibel dengan mematuhi prinsip SOLID (khususnya Single Responsibility Principle dan Dependency Inversion Principle).
4. Menghindari kebocoran memori koneksi (connection pool leak) menggunakan mekanisme Try-With-Resources serta menangani anomali eksepsi database ('SQLException') secara tepat.

BAB II: LANDASAN TEORI

2.1 Paradigma Pemrograman Berorientasi Objek (OOP)

Pemrograman Berorientasi Objek (OOP) merupakan paradigma penulisan kode program yang menyusun program berdasarkan objek-objek diskrit yang merepresentasikan entitas nyata di dunia fisik. Dalam Java, paradigma ini didukung oleh empat pilar utama:

1. Encapsulation (Pengkapsulan): Mekanisme pembungkusan data dan fungsi yang memanipulasinya di dalam suatu unit kelas. Penerapannya ditandai dengan penggunaan hak akses modifier 'private' pada variabel kelas, yang hanya dapat diakses atau diubah melalui metode publik ('getter' dan 'setter'). Ini berfungsi sebagai pelindung integritas data agar tidak diubah secara serampangan oleh modul lain.
2. Inheritance (Pewarisan): Kemampuan sebuah kelas untuk mewarisi atribut dan metode dari kelas lain. Pewarisan memfasilitasi penggunaan kembali kode (code reusability). Pada program ini, pewarisan terjadi saat kelas implementasi mengadopsi kontrak fungsional dari kelas interface (misalnya 'KamarDAOImpl' mewarisi properti interface 'KamarDAO').

3. Polymorphism (Polimorfisme): Istilah yang mendeskripsikan kemampuan suatu objek untuk mengambil berbagai bentuk tindakan. Polimorfisme memungkinkan pemanggilan metode interface yang sama namun memicu eksekusi logika konkrit yang berbeda tergantung instansiasi objek konkritnya pada saat runtime.
4. Abstraction (Abstraksi): Konsep menyembunyikan detail implementasi teknis yang rumit dari pengguna kelas, dan hanya menyajikan fitur dasar/esensial. Diwujudkan melalui pembuatan kelas interface yang bertindak sebagai kontrak formal tanpa logika SQL.

2.2 Arsitektur Sistem 3-Tier

Arsitektur 3-tier membagi sistem perangkat lunak ke dalam tiga tingkatan logis utama yang terpisah secara fisik atau logis:

- Presentation Tier (MainApp): Lapisan paling atas yang berinteraksi langsung dengan pengguna. Bertugas menerima masukan dari pengguna melalui scanner keyboard dan merender data ke konsol dalam bentuk format teks/tabel.
- Logic/Service Tier (KamarService & ReservasiService): Lapisan tengah yang mengontrol logika bisnis, perhitungan, validasi aturan hotel, koordinasi alur data, serta pengambilan keputusan logika aplikasi.
- Data Access Tier (DAO Implementation & Database): Lapisan terbawah yang bertugas menangani interaksi langsung dengan media penyimpanan persisten (database). Lapisan inilah yang menyusun query SQL dan mengeksekusinya ke DBMS.

2.3 Desain Pola Data Access Object (DAO)

Pola DAO merupakan bagian dari katalog pola desain tingkat enterprise yang mengisolasi semua operasi akses data persisten dari bagian logika bisnis aplikasi. Pola ini memisahkan interface (misalnya `KamarDAO.java`) yang mendefinisikan operasi apa saja yang tersedia, dengan kelas implementasinya (seperti `KamarDAOImpl.java`) yang menjabarkan cara melakukan operasi tersebut menggunakan driver database tertentu (JDBC MySQL). Pola ini sangat membantu ketika di kemudian hari database bermigrasi ke database NoSQL atau SQL vendor lain tanpa merusak kode program pada layer Service.

2.4 SOLID Design Principles

SOLID adalah panduan desain kelas yang sangat terkenal dalam pengembangan perangkat lunak berorientasi objek:

- Single Responsibility Principle (SRP): Menyatakan bahwa sebuah kelas hanya boleh memiliki satu tanggung jawab atau satu fokus tugas spesifik. Dengan demikian, jika terjadi perubahan pada tata cara rendering tampilan, hanya kelas `'MainApp'` yang perlu diubah. Jika query database diubah, hanya kelas `'DAOImpl'` yang perlu diubah.
- Dependency Inversion Principle (DIP): Menyarankan agar modul tingkat tinggi tidak bergantung secara langsung pada modul tingkat rendah. Keduanya harus bergantung pada abstraksi. Cara penerapannya adalah dengan melakukan injeksi dependensi (Dependency Injection) melalui parameter constructor kelas Service dengan melemparkan tipe data Interface, bukan kelas implementasinya secara langsung.

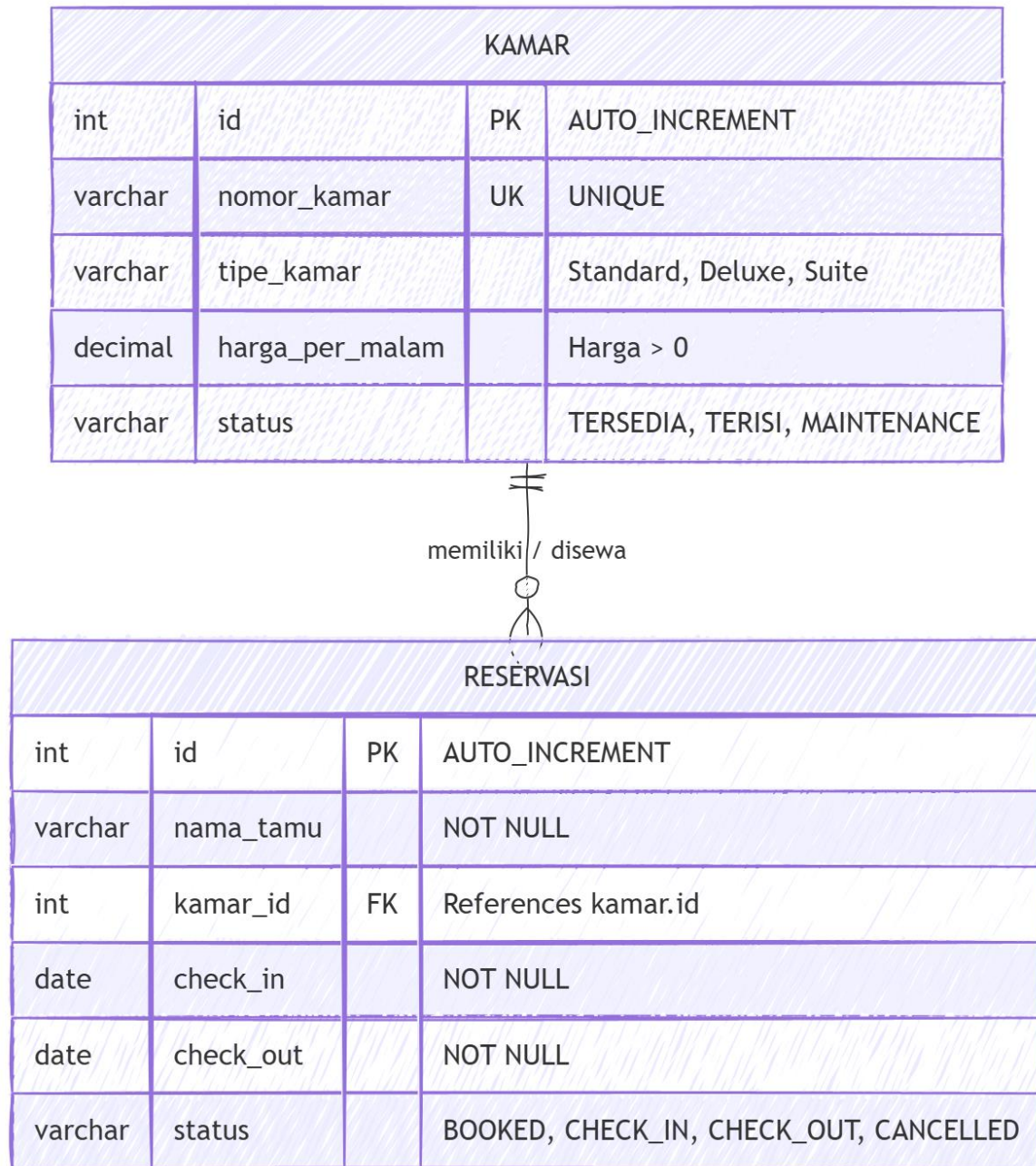
2.5 Keamanan Database Relasional menggunakan JDBC

1. PreparedStatement: Merupakan sub-interface dari `'Statement'` Java JDBC yang mengompilasi query SQL terlebih dahulu. Input dinamis dari user diikat menggunakan parameter tanda tanya ('?'), sehingga input berbahaya dari hacker (seperti `' OR '1'='1'`) akan selalu dibaca sebagai nilai string mentah biasa dan bukan sebagai instruksi SQL tambahan.
2. Try-With-Resources: Sintaksis Java modern yang mengotomatisasi penutupan objek yang mengimplementasikan interface `'AutoCloseable'` (seperti `'Connection'`, `'PreparedStatement'`, dan `'ResultSet'`). Ini menjamin bahwa koneksi ke database akan selalu ditutup di akhir blok kode, meskipun terjadi kesalahan (error/exception), membebaskan memori server database dari tumpukan koneksi mati.

BAB III: METODOLOGI DAN PERANCANGAN SISTEM

3.1 Skema Relasi Database (ERD & DDL)

Sistem ini menggunakan penyimpanan MySQL dengan struktur dua tabel yang saling terhubung melalui relasi kunci asing (Foreign Key):



- Kunci Asing `reservasi.kamar\_id` mereferensikan kolom kunci utama `kamar.id` dengan relasi satu-ke-banyak (One-to-Many).

- Aturan integritas referensial disetel ke `ON DELETE RESTRICT`, yang berarti data kamar tidak boleh dihapus secara tidak sengaja dari database jika kamar tersebut memiliki riwayat transaksi aktif di dalam tabel reservasi.

3.2 Diagram Kelas Detail (UML Class Diagram)

Struktur perancangan kode diilustrasikan secara rinci pada diagram di bawah ini:

[UML CLASS DIAGRAM](#)

3.3 Logika Validasi Alur Bisnis

Berikut adalah diagram alur logika bisnis (flowchart) yang diimplementasikan pada lapisan Service ketika tamu melakukan proses reservasi kamar:

[FLOWCHART LOGIKA VALIDASI RESERVASI](#)

BAB IV: IMPLEMENTASI DAN PENGUJIAN SISTEM

4.1 Detail Implementasi Source Code Proyek

Kode sumber lengkap proyek ditulis tanpa menggunakan baris instruksi kosong (placeholder) dan diimplementasikan pada file berikut:

[1. Kelas Model Kamar \(entity/Kamar.java\)](#)

[2. Kelas Model Reservasi \(entity/Reservasi.java\)](#)

[3. Kelas Implementasi Kamar DAO \(dao/KamarDAOImpl.java\)](#)

4.2 Laporan Hasil Pengujian Sistem (Black Box Testing)

Pengujian dilakukan menggunakan metode Black Box Testing untuk memverifikasi kesesuaian output program terhadap skenario input yang diberikan. Seluruh hasil pengujian dirangkum pada tabel berikut:

No	Modul	Skenario Input	Output yang Diharapkan	Hasil
1	Tambah Kamar	Nomor kamar = 401, Tipe = VIP, Harga = 1200000, Status = TERSEDIA	[OK] Kamar berhasil ditambahkan dengan ID: 6	BERHASIL
2	Tambah Kamar	Nomor kamar = 101 (duplikat, sudah ada di DB)	[!] Nomor kamar '101' sudah digunakan	BERHASIL
3	Tambah Kamar	Harga = -500 (negatif / nol)	[!] Harga per malam harus lebih dari 0	BERHASIL
4	Tambah Kamar	Status diisi selain TERSEDIA/TERISI/MAINTENANCE	[!] Status kamar tidak valid	BERHASIL
5	Tambah	Check-in = 2026-06-15, Check-out =	[!] Tanggal Check-Out harus	BERHASIL

	Reservasi	2026-06-10	setelah Check-In	
6	Tambah Reservasi	Kamar ID = 4 (Status MAINTENANCE)	[!] Kamar sedang dalam MAINTENANCE	BERHASIL
7	Tambah Reservasi	Kamar ID = 1 (Status TERISI / sudah dipesan)	[!] Kamar sedang TERISI	BERHASIL
8	Tambah Reservasi	Nama tamu dikosongkan	[!] Nama tamu tidak boleh kosong	BERHASIL
9	Update Reservasi	Status diubah ke CHECK_OUT	Status kamar terkait otomatis berubah ke TERSEDIA	BERHASIL
10	Hapus Reservasi	Reservasi dengan status BOOKED dihapus	Status kamar terkait otomatis kembali TERSEDIA	BERHASIL
11	Cari Kamar	Input nomor kamar yang tidak ada di DB	[!] Kamar tidak ditemukan	BERHASIL
12	Cari Reservasi	Input nama tamu sebagian (partial search)	Daftar seluruh reservasi dengan nama tamu yang mengandung keyword	BERHASIL

BAB V: KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan seluruh hasil perancangan, implementasi, dan pengujian terhadap program Hotel Reservation Management System, dapat disimpulkan bahwa:

1. Konsep dasar Pemrograman Berbasis Objek (OOP) telah diterapkan secara konsisten. Penggunaan pengkapsulan melindungi data transaksi dari manipulasi luar, sedangkan polimorfisme dan abstraksi mengizinkan penulisan kode program yang dinamis.
2. Penggunaan arsitektur berlapis (MainApp $\rightarrow$ Service Layer $\rightarrow$ DAO $\rightarrow$ Database) sangat efektif dalam membagi tanggung jawab penulisan fungsi program (High Cohesion) serta meminimalkan keterikatan kode (Low Coupling).
3. Penerapan 'PreparedStatement' berhasil menutup total kerentanan keamanan basis data dari manipulasi SQL berbahaya, sementara sintaks Try-With-Resources menjamin pemanfaatan memori koneksi server basis data secara efisien.
4. Program berhasil dieksekusi secara stabil di bawah runtime server database Laragon.

5.2 Saran

Beberapa hal yang dapat diajukan untuk meningkatkan efektivitas aplikasi ini di masa mendatang adalah:

1. Migrasi Antarmuka: Melakukan transformasi visual dengan mengganti antarmuka teks (Console) menjadi antar muka grafis (Graphical User Interface - GUI) berbasis JavaFX.
2. Penerapan Framework ORM: Mengganti JDBC murni menggunakan framework Object-Relational Mapping (ORM) seperti Hibernate untuk mempercepat penulisan baris database.
3. Sistem Multi-User: Menambahkan tabel 'user' beserta modul otentikasi login enkripsi kata sandi (SHA-256) untuk pembagian hak akses resepsionis dan manajer.